Phase 2 — Detailed Actionable Guide: Data Collection & Engineering (Week 2–3)

> **Goal:** Populate the graph schema `G = (V, E, X, W)` defined in Phase 1 with real data — extract supplier–customer relationships, pull financial metrics, resolve entity names, set up persistent storage, and build a skeleton API. By the end of this phase, you will have three critical artifacts: `supply_chain.csv`, `company_financials.csv`, and a master entity index with UUIDs — the raw ingredients that Phase 3 will transform into a live graph.

🔗 Connection from Phase 1

Phase 1 produced several outputs that Phase 2 directly consumes:

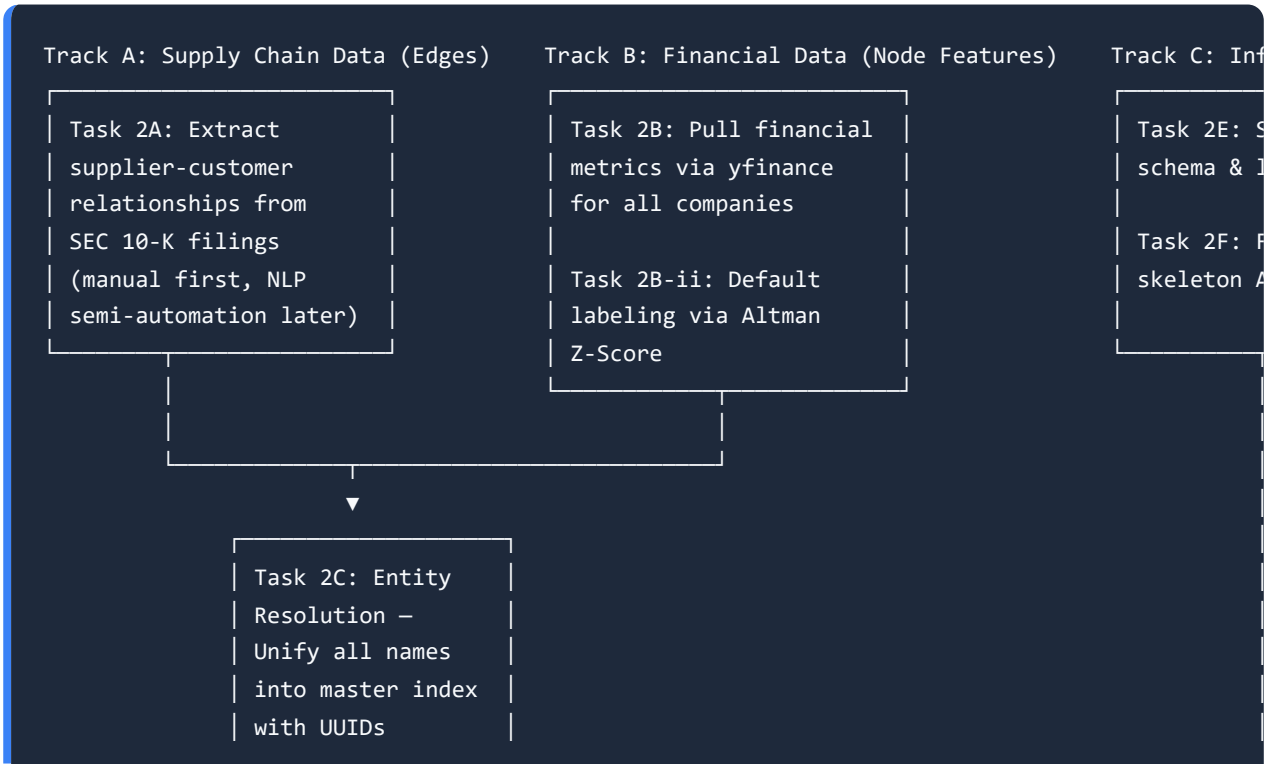
Phase 1 Output	How Phase 2 Uses It
Graph schema `G = (V, E, X, W)`	Defines the target structure you're now populating with data
30 anchor companies (`data/raw/anchor_companies.csv`)	Starting seeds for expanding to ~300 nodes via 10-K exploration
Data source evaluation matrix	Tells you which sources to use (SEC EDGAR, yfinance, Kaggle, etc.) and in what order
Default labeling strategy (Altman Z-Score)	Guides how you assign the `default_label` column in financial data

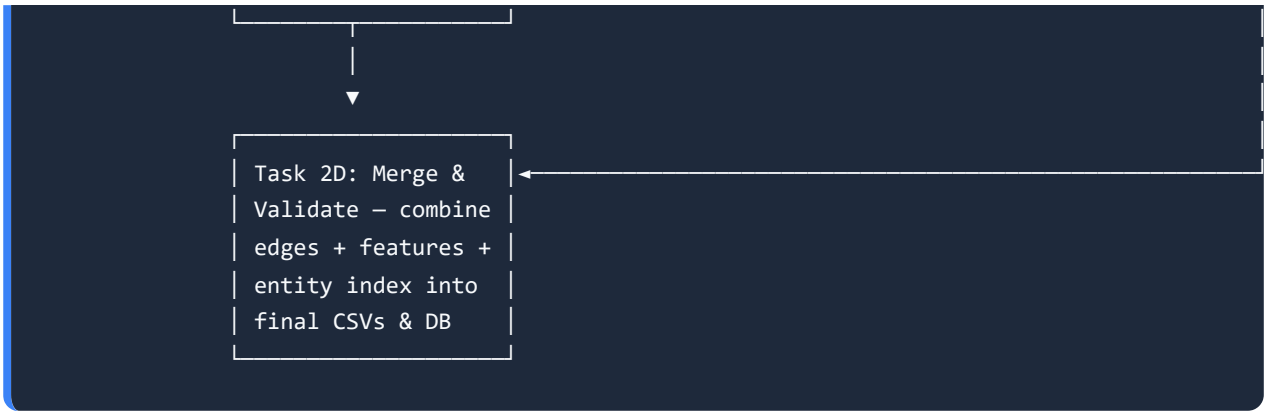
Folder structure & Git repo	You know exactly where to save every output file
Ticker verification results	You already tested which tickers work in yfinance — no wasted time

> [!IMPORTANT] **Before starting Phase 2, verify these Phase 1 artifacts exist:**
>
> - `data/raw/anchor_companies.csv` with 30 companies, tickers, tiers, and countries
> - A working conda environment (`scncc`) with `pandas` , `yfinance` , `spacy` , `fuzzywuzzy` installed
> - At least 2–3 SEC 10-K filings manually reviewed (you should already know what supplier disclosures look like)

Overview of Phase 2 Tasks

Phase 2 has **three parallel tracks** that converge at the end:





Key insight: Tracks A and B can happen **simultaneously** (two team members doing them in parallel), while Track C (infrastructure) runs independently. Task 2C (Entity Resolution) is the **critical merge point** — it cannot start until both Tracks A and B have produced initial outputs. Task 2D is the final validation step before Phase 3.

Task 2A: Supply Chain Data — Extracting Who Supplies Whom

What Are You Building?

A CSV file called `data/processed/supply_chain.csv` that contains **every known supplier–customer relationship** for your ~300 target companies. Each row represents one **directed edge** in the graph: "*Company A supplies Company B.*"

Why Is This the Hardest Part?

Unlike financial data (which comes from clean APIs), supply chain relationships are **buried inside unstructured text** in regulatory filings, press releases, and annual reports. There is no single API that gives you a clean list of "who supplies whom." You must **extract** this information through a combination of manual reading and NLP automation.

How to Do It — Step by Step

Step 2A-1: Manual Extraction from SEC 10-K Filings (Start Here, Days 1–3)

Why start manual? You need to understand the *format* and *language* of supplier disclosures before you can automate extraction. Every 10-K is different — some list

suppliers by name in tables, others mention them in sentences, and some only hint at them. By reading 10–15 filings yourself, you build the intuition needed to design effective NLP pipelines.

Exact process for each anchor company:

1. **Go to SEC EDGAR full-text search:** efts.sec.gov/LATEST/search-index
2. **Search by company name or CIK number** (e.g., search "Ford Motor")
3. **Open the latest 10-K filing** (annual report, not 10-Q which is quarterly)
4. **Navigate to the key sections where supplier/customer info lives:**

10-K Section	What You'll Find	Priority
Item 1 — Business	Overview of operations, major suppliers, manufacturing, key customers	🔴 Most Important
Item 1A — Risk Factors	"concentration risk", "sole source supplier", "key customer" mentions	🔴 Very Important
Item 7 — MD&A	Management discussion — mentions of supply chain disruptions, key partner impacts	🟡 Important
Item 8 — Financial Statements (Notes)	Revenue by segment, major customer disclosures (ASC 280)	🟡 Important

5. **Use Ctrl+F to search for these keywords** within each section:
   - `"major supplier"`, `"significant supplier"`, `"key supplier"`, `"sole source"`
   - `"principal customer"`, `"major customer"`, `"significant customer"`
   - `"supply agreement"`, `"purchase commitment"`, `"raw material"`
   - `"accounts for approximately"` (often precedes revenue concentration disclosures)
6. **For every supplier/customer pair you find, record it in a row:**

```
supplier,customer,relationship_type,data_source,confidence,year,notes
Denso Corp,Toyota Motor,tier1_supplier,sec_10k_TM_2024,1.0,2024,"thermal systems a
```

What each column means:

Column	Description	Why You Need It
`supplier`	The supplying company's name (as written in the filing)	Raw name — will be resolved to UUID in Task 2C
`customer`	The receiving company's name	Raw name — will be resolved to UUID in Task 2C
`relationship_type`	`tier1_supplier`, `tier2_supplier`, `trade_credit`, `ownership`	Maps to edge types in the graph schema from Phase 1
`data_source`	Where you found this info (e.g., `sec_10k_F_2024`)	Provenance tracking — essential for later auditing and confidence scoring
`confidence`	How sure you are this relationship exists (0.0–1.0)	`1.0` for explicitly named in filing; `0.7` for NLP-extracted; `0.5` for inferred
`year`	Which fiscal year the filing covers	Edges may expire — a supplier from 2018 may not be relevant in 2024
`notes`	Any context about what is being supplied	Useful for edge weight estimation later (Phase 3)

> [!TIP] **Target for manual extraction:** Get 50–80 edges from the first 10–15 anchor companies. Don't try to be exhaustive — this is about building intuition and seeding the dataset. NLP will scale it later.

Step 2A-2: Semi-Automated NLP/NER Extraction (Days 3–7)

Why semi-automate? Manual extraction doesn't scale to 300 companies. NLP (Natural Language Processing) with NER (Named Entity Recognition) can read filings and automatically extract organization names. It won't be perfect, so you'll need to manually verify outputs — hence "semi-automated".

How it works — the pipeline in plain English:

1. **Download the SEC filing** (it's in HTML format)
2. **Convert HTML → plain text / Markdown** (strip formatting noise, preserve structure like headings and tables — reduces token count for NLP processing)
3. **Run a spaCy NER model** over the text to find all mentions of organization names (labeled `ORG`)
4. **Filter to relevant sections** (Item 1, 1A, 7 only — not the entire filing) to reduce false positives
5. **Match detected ORGs with known company names** using fuzzy matching (Task 2C logic)
6. **Output new edges** for human review

What is spaCy? spaCy is a Python library for NLP. Its NER component can read text and tag words with categories like `ORG` (organization), `PERSON`, `GPE` (geographic), etc. It's not custom-trained for SEC filings, so it will have some errors — which is why human review is needed.

What is NER? Named Entity Recognition is the task of finding and categorizing real-world entities mentioned in text. For example, in the sentence "*Denso Corporation supplies thermal systems to Toyota Motor*", NER identifies `Denso Corporation` and `Toyota Motor` as `ORG` entities.

Detailed pipeline steps:

1. **HTML to Markdown conversion:**
    - SEC filings are filed in HTML format (or XBRL, which can be converted)
    - You need to strip out HTML tags, scripts, and styling while keeping headings, tables, and paragraph structure
    - Tools: Python's `beautifulsoup4` library for HTML parsing, or `markdownify` library to convert HTML → Markdown

- **Why Markdown?** Markdown is much smaller than HTML (fewer tokens for NLP), preserves structural information (headings help identify which sec filing section you're in), and is human-readable for manual review

2. **Section Isolation:**
   - Not all sections of a 10-K contain supplier info. Extract **only** Items 1, 1A, and 7
   - Use heading detection (look for patterns like `"Item 1."`, `"ITEM 1"`, `"Item 1A"` in the Markdown)
   - This drastically reduces false positives — the financial tables in Item 8 contain many company name mentions that are NOT supply chain relationships

3. **NER Application:**
   - Load spaCy's English model: `nlp = spacy.load("en_core_web_sm")` (small model, fast) or `en_core_web_trf` (transformer-based, more accurate but slower)
   - Process each section: `doc = nlp(section_text)`
   - Extract ORG entities: `[ent.text for ent in doc.ents if ent.label_ == "ORG"]`
   - **Expect false positives:** spaCy will tag things like "Board of Directors", "SEC", "NYSE" as ORGs. You'll need to filter these out using a stoplist (list of organization names that are NOT supply chain participants)

4. **Context-Aware Extraction (optional, advanced):**
   - Instead of just finding ORG names, look for ORG names that appear **near** supply-chain keywords
   - For example: find all ORGs within 50 words of `"supplier"`, `"provides"`, `"manufactures for"`, `"sole source"`, `"purchases from"`
   - This dramatically improves precision (fewer false positives) at the cost of some recall (might miss obliquely referenced suppliers)

5. **Output for Review:**
   - Output a CSV of detected relationships with the section context around them
   - A human reviewer approves, edits, or rejects each entry
   - Approved entries go into `supply_chain.csv`

> [!WARNING] **spaCy's default NER model is NOT trained on SEC filings.** It will miss some companies and incorrectly tag others. Accuracy is typically 60–75% on financial text out of the box. This is acceptable for seeding your edge list — you compensate by manual review and by using multiple data sources.

Step 2A-3: Supplementary Data Sources (Parallel with NLP, Days 3–7+)

Why use multiple sources? No single source captures all supply chain relationships. SEC filings are high-confidence but only cover US public companies' major relationships. Supplementary sources fill gaps.

Source	What You Get	How to Access	What to Watch Out For
Kaggle datasets	Pre-built supply chain networks (variable quality)	Search `"supply chain network"` on kaggle.com, download CSVs	May not match your sector; verify entity names match your index
FactSet Revere	Curated supplier-customer pairs with revenue dependency ratios	Paid; some universities have access; try free trial	Excellent data but paywalled
Panjiva / ImportGenius	US customs records — shipment-level trade data between companies	Paid subscription	Raw customs data needs ML parsing to extract company names and aggregate into edge weights
CSMAR	Auto supplier lists for Chinese market companies	Institutional access (often available at Chinese universities)	Coverage limited to Asia-Pacific

Company annual reports	Specifically for private companies (Bosch, ZF, Yazaki) that don't file with SEC	Google `"Bosch annual report 2024 PDF"` → download → read manually	No standardized format; fully manual extraction
Fed Reserve / Crunchbase	Cross-validation — check if your extracted relationships match known datasets	Free (Crunchbase basic tier)	Crunchbase focuses on investment, not supply chains — limited overlap

How to integrate supplementary data:

1. Download / extract edges from each source into the same CSV format
2. Add the source name to the `data_source` column (e.g., `kaggle_xyz`, `factset_revere`)
3. Adjust the `confidence` column (e.g., `1.0` for FactSet Revere, `0.6` for Kaggle due to uncertain provenance)
4. All names go through entity resolution (Task 2C) to merge with your master index

Step 2A-4: Expanding from 30 to 300 Companies

Why expand? Your 30 anchors are the seeds, but the graph needs ~300 nodes and ~1,000 edges to be meaningful for GNN training. Expansion happens organically as you extract relationships.

How it works — the snowball process:

1. Start with your 30 anchor companies
2. Extract their suppliers and customers from 10-K filings
3. Many of these suppliers/customers are **new companies** not in your original list
4. Add these new companies to your entity index
5. If the new company is public and has a 10-K filing, repeat the process for their filing
6. After 2–3 rounds of snowballing, you'll have 200–300+ companies

Tier structure that emerges:

```
OEMs (Tier 0, your anchor seeds)
  └── Tier 1 Suppliers (directly supply OEMs — Denso, Bosch, Continental)
```

```
        └── Tier 2 Suppliers (supply Tier 1 — specialty materials, sub-components)
              └── Tier 3 / Raw Materials (steel, aluminum, chemicals — Nucor, BASF)
```

> [!NOTE] **Data visibility drops sharply at Tier 2+.** Most Tier 2 and Tier 3 suppliers are private companies with no SEC filings and limited public data. You may only know their name but not their financials. This is a known limitation that Phase 4 addresses with CuGNN (contrastive learning to infer latent dependencies from structural patterns).

Practical target: Aim for **30 OEMs, 80–100 Tier 1, 100–120 Tier 2, 50–70 Tier 3/Raw Material** companies, with ~1,000 edges connecting them.

Task 2B: Financial Data — Company-Level Node Features

What Are You Building?

A CSV file called `data/processed/company_financials.csv` that contains **financial metrics for every company in your graph**. These metrics become the **intrinsic node features** `X_i` that the GNN learns from — they represent each company's individual financial health **before** any network effects.

Why Are These Specific Metrics Chosen?

Each metric was chosen because it captures a different dimension of a company's vulnerability to credit distress:

Feature	What It Measures	Why It Matters for Contagion
`revenue`	Company scale (total annual revenue)	Larger companies have more capacity to absorb shocks; smaller ones are more fragile

`total_debt`	Total debt outstanding	High debt makes a company more vulnerable to liquidity crises if a customer defaults
`market_cap`	Market's valuation (share price × shares)	Market's real-time assessment of the company's health; drops signal distress faster than accounting metrics
`current_ratio`	Current Assets / Current Liabilities	**Liquidity** — can the firm pay its bills in the next 12 months? Low values (< 1.0) = liquidity stress
`debt_to_equity`	Total Debt / Shareholder Equity	**Leverage** — how much of the company is funded by debt vs. equity? High leverage = amplified risk
`profit_margin`	Net Income / Revenue	**Profitability** — is the firm making money? Negative margins = burning cash
`revenue_growth`	Year-over-year % change in revenue	**Trajectory** — is the firm growing or shrinking? Shrinking firms are more vulnerable
`interest_coverage`	EBIT / Interest Expense	**Debt servicing ability** — can the firm afford its debt payments? Values < 1.5 are dangerous
`roa`	EBIT / Total Assets	**Asset efficiency** — how well does the firm use its assets to generate profit?
`altman_z_score`	Composite bankruptcy predictor (see below)	Directly predicts default probability — see Phase 1 Z-Score strategy

How to Do It — Step by Step

Step 2B-1: Pull Data via yfinance API (Days 1–3)

For each public company in your anchor list (and newly discovered companies from Task 2A):

1. Use the `yfinance` Python library to fetch financial data
2. The `stock.info` dictionary contains most of the fields you need
3. Some fields may return `None` — handle these gracefully with defaults or manual lookup

The `get_financial_features()` function explained:

- **Input:** A stock ticker string (e.g., `"TM"` for Toyota, `"F"` for Ford)
- **Process:** Calls the yfinance API, which fetches the latest financial summary from Yahoo Finance
- **Output:** A dictionary with all the financial metrics listed above
- **Why a function?** You'll call this ~300 times (once per company), so wrapping it in a function avoids code duplication and makes the pipeline repeatable

What happens inside `yf.Ticker(ticker).info`:

- yfinance sends an HTTP request to Yahoo Finance's API
- Yahoo returns a JSON object with ~150 financial fields
- You extract only the ~10 fields you need for the feature matrix
- Some fields (like `interestCoverage`) are not directly available and must be computed from others

Handling missing data (critical for data quality):

Scenario	What to Do	Why
Ticker returns `None` for all fields	Try alternate ticker format (e.g., `7203.T` for Toyota on Tokyo exchange instead of `TM` ADR)	Different exchanges have different data availability
Individual field is `None` (e.g., `debtToEquity`)	Set to `NaN` and note it; later impute using sector median	Dropping entire companies for missing fields loses valuable graph structure
Company is private (no ticker)	Manually extract from annual reports (PDF)	Private companies (Bosch, ZF, Yazaki) are essential Tier 1 nodes — you can't skip them

Revenue is 0 or negative	Double-check the ticker; may be a holding company with revenue reported at subsidiary level	False zeros will distort normalization in Phase 3

> [!CAUTION] **yfinance data is delayed and occasionally inaccurate.** Always spot-check a few values against actual SEC filings or the company's investor relations page. If a metric seems wildly wrong (e.g., Toyota's revenue showing as $0), the API call may have failed silently.

Step 2B-2: Compute Altman Z-Score (Days 3–4)

What is the Altman Z-Score? It's a well-established formula invented by NYU professor Edward Altman in 1968 that combines five financial ratios into a single number predicting the probability of bankruptcy within 2 years.

The formula:

$$Z = 1.2 \cdot X_1 + 1.4 \cdot X_2 + 3.3 \cdot X_3 + 0.6 \cdot X_4 + 1.0 \cdot X_5$$

What each component measures and how to compute it:

Component	Formula	What It Captures	How to Get the Data
X_1	Working Capital / Total Assets	**Liquidity** — short-term financial health	`working_capital = current_assets - current_liabilities` from balance sheet
X_2	Retained Earnings / Total Assets	**Cumulative Profitability** — has the firm been profitable over time?	`retained_earnings` from balance sheet
X_3	EBIT / Total Assets	**Operating Efficiency** — profit from core	`EBIT` from income statement

		operations relative to assets	
X_4	Market Cap / Total Liabilities	**Solvency** — market's valuation relative to obligations	`market_cap` from yfinance, `total_liabilities` from balance sheet
X_5	Revenue / Total Assets	**Asset Turnover** — how efficiently assets generate revenue	Both from financial statements

Getting the underlying data for Z-score computation:

- `stock.balance_sheet` → gives Total Assets, Current Assets, Current Liabilities, Retained Earnings, Total Liabilities
- `stock.income_stmt` → gives EBIT (or Operating Income), Revenue
- `stock.info['marketCap']` → gives Market Capitalization
- Some of these come as DataFrames with multiple years — use the **most recent year** (first column)

Interpretation thresholds — the labeling rules:

Z-Score Range	Zone	Meaning	Your `default_label`
Z > 2.99	**Safe Zone**	Financially healthy, very low default risk	`0`
1.81 < Z ≤ 2.99	**Grey Zone**	Uncertain — could go either way	`0` (but flag for manual review)
Z ≤ 1.81	**Distress Zone**	High bankruptcy risk within 2 years	`1`

Why these thresholds? Altman validated them on a dataset of 66 companies (33 bankrupt, 33 healthy). The 1.81 and 2.99 cutoffs achieved 95% classification accuracy in the original study. While the model is from 1968, it remains one of the most widely used bankruptcy predictors in finance.

> [!NOTE] **Expected label distribution:** For your ~300 company dataset, you should expect roughly 5–15% to fall in the Distress Zone (Z ≤ 1.81). If you get 0% defaults, your dataset is too biased toward healthy megacaps. If you get > 30%, something is wrong with your Z-score computation. Aim for at least 15–30 companies labeled as `default_label = 1` — this gives the GNN enough positive examples to learn from.

Step 2B-3: Handle Private Companies (Days 4–5)

The problem: Three of your anchor companies (**Robert Bosch**, **ZF Friedrichshafen**, **Yazaki**) are private and don't have stock tickers. As you expand via snowballing, you'll discover more private companies. They have no yfinance data.

The solution: Manual data extraction from public sources:

Source	How to Access	What You Get
Annual reports	Google `"Bosch annual report 2024"` → download PDF	Revenue, EBIT, total assets, debt, employees
Press releases	Company website → Investor Relations / Media	Revenue announcements, major contract wins
Financial databases	Bloomberg Terminal (if university has access)	Full financials
Dun & Bradstreet	Paid, but limited free data	Revenue estimates
Industry reports	Google Scholar, McKinsey, Deloitte	Estimated market share, revenue ranges

What to do if you can't find exact financial data:

1. Estimate using **sector averages** — if Bosch is in "automotive electronics" and the sector average profit margin is 6%, use 6%

2. Set the `confidence` column to a lower value (e.g., `0.5` for estimated data vs. `1.0` for verified API data)
3. **Do NOT drop private companies from the graph** — they are often critical Tier 1 suppliers. A node with estimated features is far better than no node at all, because the graph structure (edges) carries most of the contagion signal

Step 2B-4: Final Financial Dataset Structure

Your output file `data/processed/company_financials.csv` should look like:

```
entity_id,name,ticker,revenue,total_debt,market_cap,current_ratio,debt_to_equity,profit_r
,Toyota Motor,TM,280000000000,200000000000,250000000000,1.1,1.2,0.07,0.05,8.5,0.04,3.45,0
,Robert Bosch,,78000000000,25000000000,,1.3,1.5,0.05,0.03,5.2,0.03,2.8,0,annual_report_20
,ElectroParts,ELCP,1000000000,500000000,800000000,0.6,3.5,0.01,-0.10,0.8,0.01,1.2,1,yfina
```

- `entity_id` is left blank for now — assigned during entity resolution (Task 2C)
- `data_source` tracks where each company's data came from
- `data_confidence` helps Phase 3 weigh feature reliability

Task 2C: Entity Resolution — The Critical Merge Step

What Is Entity Resolution?

Entity resolution is the process of determining that **different name strings refer to the same real-world entity**. It's the most critical data engineering step in Phase 2.

Why Is This Necessary?

You now have supplier names from SEC filings (Task 2A) and financial data from yfinance (Task 2B), but the **same company appears differently across your sources:**

Source	How They Write It
Ford's 10-K filing	"Continental AG"
Toyota's 10-K filing	"Continental Automotive"

yfinance API	"Continental Aktiengesellschaft"
Kaggle dataset	"CONTINENTAL"
Another filing	"Continental, tires and automotive"

All five of these are the **same company** — Continental AG (Ticker: `CTTAY`). If you don't resolve them, your graph will have **five separate nodes** for what should be one node, and the edges from each source won't connect properly.

How to Do It — Multi-Strategy Approach

Why multiple strategies? No single matching technique catches everything. Exact string matching misses "Continental AG" vs. "Continental Automotive". Fuzzy matching might incorrectly merge "BAE Systems" and "BASF" (high character similarity, different companies). Combining strategies gives the best accuracy.

Strategy 1: Deterministic Matching (Normalization)

What: Convert all names to a canonical form before comparing.

Steps:

1. Convert to uppercase: `"Continental AG"` → `"CONTINENTAL AG"`
2. Remove common suffixes: `"Corp"`, `"Corporation"`, `"Inc"`, `"Ltd"`, `"AG"`, `"GmbH"`, `"Co."`, `"S.A."`, etc.
3. Remove punctuation and extra whitespace
4. Remove country identifiers in parentheses

After normalization:

- `"Continental AG"` → `"CONTINENTAL"`
- `"Continental Automotive"` → `"CONTINENTAL AUTOMOTIVE"` (still different — needs Strategy 2)

Strategy 2: Probabilistic Matching (Fuzzy String Similarity)

What: Compare normalized names using string similarity algorithms. Use the `fuzzywuzzy` library.

How `fuzz.ratio()` works:

- Compares two strings character by character
- Returns a score from 0 (completely different) to 100 (identical)
- Uses the Levenshtein distance (number of single-character edits needed to transform one string into the other)

Threshold selection:

Similarity Score	Action	Example
≥ 90	**Auto-merge** — very likely the same entity	"CONTINENTAL" vs "CONTINENTAL AUTOMOTIVE" (depends on token ratio)
80–89	**Flag for manual review** — might be the same	"HYUNDAI MOTOR" vs "HYUNDAI MOBIS" (related but different companies!)
< 80	**Assume different** — different entities	"BASF" vs "BAE SYSTEMS"

> [!WARNING] **Watch out for false merges!** "Hyundai Motor" and "Hyundai Mobis" are two DIFFERENT companies (parent and subsidiary). Fuzzy matching will give them a high score (~85). Always use `fuzz.token_sort_ratio()` (which sorts tokens before comparing) AND verify against ticker/country data when available. If two names resolve to different tickers, they are different entities regardless of name similarity.

Recommended fuzzy matching function: `fuzz.token_sort_ratio()` instead of `fuzz.ratio()`

- `fuzz.ratio("Continental AG", "AG Continental")` → low score (word order differs)
- `fuzz.token_sort_ratio("Continental AG", "AG Continental")` → 100 (ignores word order)

Strategy 3: Ticker-Based Matching (Most Reliable)

What: If two name variants resolve to the same stock ticker, they are definitively the same entity.

How:

1. When you extract a name from a filing, check if it matches a known ticker in your anchor list
2. Use yfinance to search: `yf.Ticker("CTTAY").info['shortName']` → "Continental Aktiengesellschaft"
3. If the official name matches your extracted name, assign them the same UUID

Why this is the most reliable: Tickers are unique identifiers assigned by stock exchanges. Two different companies cannot have the same ticker. If an extracted name resolves to a known ticker, the match is certain.

Strategy 4: Graph-Based Matching (Advanced, Optional)

What: If two name variants share many of the same neighbors (suppliers or customers), they are likely the same entity — even if their names don't match well.

How:

- Build a preliminary graph from your edge list
- For each pair of nodes with similarity score 70–89 (ambiguous zone), check if they share > 50% of their neighbors
- If yes → likely the same entity (merge them)
- If no → likely different entities

Why this works: It's unlikely that two different companies would have the exact same set of suppliers AND customers. Shared neighbors are strong evidence of entity identity.

The Master Entity Index — Output of Task 2C

Your output is a lookup dictionary (saved as `data/processed/entity_index.json` or `.csv`) that maps **every raw name variant** to a **single canonical UUID:**


```
{
  "Continental AG": "a3b8d1b6-0b3b-4b1a-9c1a-1a2b3c4d5e6f",
  "Continental Automotive": "a3b8d1b6-0b3b-4b1a-9c1a-1a2b3c4d5e6f",
  "Continental Aktiengesellschaft": "a3b8d1b6-0b3b-4b1a-9c1a-1a2b3c4d5e6f",
  "CONTINENTAL": "a3b8d1b6-0b3b-4b1a-9c1a-1a2b3c4d5e6f",
  "Toyota Motor": "c5dae3d8-2d5d-6d3c-bd3c-3c4d5e6f7a8b",
```

```
  "Toyota Motor Corporation": "c5dae3d8-2d5d-6d3c-bd3c-3c4d5e6f7a8b",
  "TOYOTA": "c5dae3d8-2d5d-6d3c-bd3c-3c4d5e6f7a8b"
}
```

Why UUIDs? A UUID (Universally Unique Identifier) is a 128-bit random string that is guaranteed to be unique. Using UUIDs instead of names as node IDs means:

- No collision even if two companies have similar names
- All data sources reference the same ID for the same company
- The ID is stable even if the company changes its name

After entity resolution, go back and **fill in the `entity_id` column** in both `supply_chain.csv` and `company_financials.csv` using this index.

Task 2D: Merge, Validate, and Quality-Check

What Is This Step?

Before handing your data to Phase 3 (graph building), you need to ensure that Tasks 2A, 2B, and 2C produced consistent, clean data. This is a **validation checkpoint**.

Validation Checks to Run

Check 1: Referential Integrity

What: Every company mentioned in `supply_chain.csv` (as supplier or customer) must have a matching row in `company_financials.csv`.

Why: If an edge references a company with no financial data, that node in the graph will have no features — the GNN can't learn from empty feature vectors.

How to check:

- Get the set of all unique company names in `supply_chain.csv` (both supplier and customer columns)
- Get the set of all unique company names in `company_financials.csv`
- The difference set = companies with edges but no financial data

- For each missing company: either collect their financial data (Task 2B) or remove their edges

Check 2: Orphan Nodes

What: Companies in `company_financials.csv` that have NO edges in `supply_chain.csv`.

Why: A node with no edges is disconnected from the graph — it can't transmit or receive contagion, so it adds nothing to the model. It wastes computation.

How to handle:

- If it's an anchor company → you missed extracting their relationships. Go back to Task 2A
- If it's a newly discovered company with only one connection → keep it (even one edge is valuable)
- If it's truly isolated → remove from the dataset (or keep it but note it will be ignored by the GNN)

Check 3: Entity Resolution Completeness

What: No duplicate nodes — every company should have exactly one UUID in the entity index.

How to check:

- Group `company_financials.csv` by `entity_id` — if any UUID appears more than once, you have merged two different companies into one (false merge) or failed to merge two records of the same company
- Check `supply_chain.csv` for edges that connect a company to itself (self-loops) — these indicate entity resolution errors (two names for the same company weren't merged)

Check 4: Data Completeness

What: How many `NaN` or `0` values are in your financial data?

Accept if: < 20% of values are missing across all features for all companies **Investigate if:** > 20% missing — the GNN's performance will degrade with too many imputed values

Missing data strategy for Phase 3:

- `NaN` values will be imputed using **sector median** (e.g., if Toyota's `interest_coverage` is missing, use the median for all Automotive OEMs)
- This is better than using 0 (which falsely signals distress) or dropping the company (which loses graph structure)

Check 5: Label Distribution

What: What percentage of companies have `default_label = 1`?

Distribution	Status	Action
5–15% defaults	✅ Ideal	Proceed to Phase 3
< 5% defaults	⚠️ Too few positives	Lower your Z-Score threshold slightly, or add credit rating downgrades as additional labels
> 30% defaults	⚠️ Too many positives	Check your Z-Score computation for errors; your dataset may be biased toward small/distressed companies

Check 6: Edge Direction Consistency

What: Verify that edge directions follow the schema from Phase 1.

- `supply` edges: Supplier → Customer (A supplies B)
- `trade_credit` edges: Creditor → Debtor
- `ownership` edges: Parent → Subsidiary

Why this matters: The GNN's message passing respects edge direction. If edges are reversed, contagion will flow in the wrong direction — upstream instead of downstream or vice versa.

Task 2E: SQLite Database Setup

What Are You Building?

A lightweight database (`data/scncc.db`) that stores the same data as your CSVs, but in a structured, queryable format. This database will be consumed by the FastAPI backend (Task 2F) to serve data to the frontend.

Why Use a Database When You Already Have CSVs?

Feature	CSV Files	SQLite Database
Query flexibility	Must load entire file into Pandas	SQL queries — filter, join, aggregate on demand
Concurrent access	File locking issues if API reads while you write	Built-in transaction support
Data integrity	No constraints — can insert invalid data	Foreign keys, types, NOT NULL constraints
API integration	Must load CSV on every request	Database connection is persistent, fast
Size	Fine for ~300 rows	Fine for ~300 rows (SQLite handles millions)

For a prototype with ~300 companies, CSVs would technically work. But using SQLite from Phase 2 means your API endpoints (Phase 3 and beyond) don't need refactoring when data grows.

Database Schema — Two Tables

Table 1: `companies` — stores node data (one row per company)

Column	SQLite Type	Constraint	Maps to Graph Schema
`id`	TEXT	PRIMARY KEY	`entity_id` (UUID from Task 2C)
`name`	TEXT	NOT NULL	Canonical company name

`ticker`	TEXT	nullable	Stock ticker (NULL for private companies)
`sector`	TEXT		Industry sector
`tier`	TEXT		`OEM`, `Tier1`, `Tier2`, `Tier3`, `RawMaterial`
`country`	TEXT		HQ country code
`is_public`	INTEGER		1 = public, 0 = private
`revenue`	REAL		From yfinance / annual reports
`total_debt`	REAL		
`market_cap`	REAL		
`current_ratio`	REAL		
`debt_to_equity`	REAL		
`profit_margin`	REAL		
`revenue_growth`	REAL		
`interest_coverage`	REAL		
`roa`	REAL		
`altman_z_score`	REAL		From Task 2B-2 computation
`default_label`	INTEGER	DEFAULT 0	0 = healthy, 1 = distressed

Table 2: `relationships` — stores edge data (one row per relationship)

Column	SQLite Type	Constraint	Maps to Graph Schema
`id`	INTEGER	PRIMARY KEY AUTOINCREMENT	Internal row ID

`supplier_id`	TEXT	FOREIGN KEY → companies.id	Source node UUID
`customer_id`	TEXT	FOREIGN KEY → companies.id	Target node UUID
`relationship_type`	TEXT		`supply` , `trade_credit` , `ownership`
`weight`	REAL	DEFAULT 1.0	Edge weight (transaction intensity)
`confidence`	REAL	DEFAULT 1.0	How certain is this edge?
`data_source`	TEXT		Provenance (e.g., `sec_10k_F_2024`)
`year`	INTEGER		Fiscal year the relationship was observed

How to Build — Step by Step

1. **Create the database and tables** using the SQL `CREATE TABLE` statements from the roadmap
2. **Write a Python loader script** (`api/load_data.py`) that:
   - Reads `data/processed/company_financials.csv`
   - Reads `data/processed/supply_chain.csv`
   - Inserts rows into the `companies` and `relationships` tables
   - Uses `entity_id` from the master entity index as the primary key for companies
   - Uses `entity_id` from the entity index for `supplier_id` and `customer_id` in relationships
3. **Verify** by running a few SQL queries to check data was loaded correctly:
   - `SELECT COUNT(*) FROM companies;` → should return ~300
   - `SELECT COUNT(*) FROM relationships;` → should return ~1,000
   - `SELECT * FROM companies WHERE default_label = 1;` → should return 15–30 distressed companies
   - `SELECT r.*, c1.name as supplier_name, c2.name as customer_name FROM relationships r JOIN companies c1 ON r.supplier_id = c1.id JOIN`

`companies c2 ON r.customer_id = c2.id LIMIT 10;` → verify edges connect to real companies

> [!NOTE] **SQLite is temporary.** For production or if your graph grows beyond ~10,000 nodes, you'll migrate to **PostgreSQL** or a graph database like **Neo4j**. For now, SQLite is perfect — zero setup, file-based, and fast enough for prototyping.

Task 2F: FastAPI Skeleton Setup

What Are You Building?

A minimal Flask-like web API server using FastAPI that will eventually serve your graph data, model predictions, and simulation results to the frontend dashboard. In Phase 2, you only set up the **skeleton** — a running server with a health-check endpoint.

Why Set It Up Now?

Building the API skeleton in Phase 2 means:

- Phase 3 can immediately create a `GET /graph` endpoint (no setup delay)
- Phase 4 can add a `POST /predict` endpoint
- Phase 5 can add a `POST /simulate` endpoint
- The frontend developer can start building UI components against a live API from Day 1

What FastAPI Is

FastAPI is a modern Python web framework for building REST APIs. It:

- Automatically generates API documentation (visit `/docs` for an interactive Swagger UI)
- Uses Python type hints for request/response validation
- Supports async operations (useful for long-running GNN predictions)
- Runs on `uvicorn` (a fast ASGI web server)

How to Set Up — Step by Step

1. **Install FastAPI and uvicorn** (should already be in your conda environment):

```
pip install fastapi uvicorn
```

2. **Create the main API file:** `api/main.py`
   - Define a FastAPI app instance with a descriptive title
   - Create a root `/` endpoint that returns a simple JSON health check
   - This is the foundation that all future endpoints build on

3. **Run the server:**

```
uvicorn api.main:app --reload
```

   - `api.main:app` tells uvicorn to find the `app` object in `api/main.py`
   - `--reload` enables hot-reloading (server restarts when you edit code — development only)
   - By default, the server runs on `http://localhost:8000`

4. **Verify it works:**
   - Open `http://localhost:8000` in your browser → should see `{"status": "running", "message": "Supply Chain Contagion API"}`
   - Open `http://localhost:8000/docs` → should see Swagger UI with your endpoint listed

Endpoints You'll Eventually Need (Plan Ahead, Build Later)

Endpoint	Phase	What It Does
`GET /`	Phase 2 (now)	Health check
`GET /graph`	Phase 3	Returns full graph as JSON (nodes + edges)
`GET /companies/{id}`	Phase 3	Returns one company's details + risk scores

`POST /predict`	Phase 4	Accepts company → returns GNN-predicted default probability
`POST /simulate`	Phase 5	Runs shock simulation → returns before/after PDs
`GET /rankings`	Phase 5	Returns systemic risk rankings

🔗 Connection to Phase 3 — What Happens Next

Phase 2 ends with these deliverables:

Deliverable	File	What Phase 3 Does With It
Supply chain edges	`data/processed/supply_chain.csv`	Loaded into NetworkX as directed edges `(u, v)`
Financial features	`data/processed/company_financials.csv`	Becomes node feature matrix `X ∈ ℝ^{n × d}`
Master entity index	`data/processed/entity_index.json`	Ensures node IDs are consistent across edge list and feature matrix
SQLite database	`data/scncc.db`	Backend for API queries
FastAPI skeleton	`api/main.py`	Gets `GET /graph` endpoint added to serve graph data to frontend

The critical handoff: Phase 3 reads `supply_chain.csv` and creates `G = nx.DiGraph()` by calling `G.add_edge(supplier_id, customer_id, ...)` for every row. It then reads `company_financials.csv` and assigns each company's financial metrics as **node attributes**. Finally, it computes **topological features** (degree centrality, betweenness, eigenvector centrality, PageRank, clustering coefficient) that augment the node feature

matrix from 10 dimensions (financial) to 16 dimensions (financial + topological). This augmented graph `G = (V, E, X, W)` is what the GNN trains on in Phase 4.

Phase 2 Completion Checklist

By end of Week 3, you should have **all** of these done:

- ☐ **Task 2A:** `data/processed/supply_chain.csv` with ~1,000 edges
  - ☐ Manual extraction from 10–15 SEC 10-K filings completed
  - ☐ spaCy NER pipeline tested on at least 5 filings
  - ☐ Supplementary sources (Kaggle, annual reports) checked and integrated
  - ☐ Snowball expansion: 30 anchor companies → ~300 companies discovered
- ☐ **Task 2B:** `data/processed/company_financials.csv` with ~300 companies
  - ☐ yfinance pull completed for all public companies
  - ☐ Private companies (Bosch, ZF, Yazaki, etc.) manually added from annual reports
  - ☐ Altman Z-Score computed for every company
  - ☐ `default_label` assigned based on Z-Score thresholds
- ☐ **Task 2C:** `data/processed/entity_index.json` with UUID mappings
  - ☐ All name variants resolved to canonical UUIDs
  - ☐ No duplicate nodes (verified)
  - ☐ `entity_id` column filled in both CSVs
- ☐ **Task 2D:** Validation checks passed
  - ☐ Referential integrity: all edge endpoints have financial data
  - ☐ No orphan nodes (or documented why they exist)
  - ☐ Label distribution: 5–15% default rate
  - ☐ Edge directions consistent with schema
- ☐ **Task 2E:** `data/scncc.db` SQLite database
  - ☐ `companies` and `relationships` tables created
  - ☐ Data loaded from CSVs
  - ☐ Verification queries pass
- ☐ **Task 2F:** FastAPI skeleton
  - ☐ `api/main.py` created with health-check endpoint
  - ☐ Server runs successfully on `localhost:8000`
  - ☐ Swagger docs accessible at `/docs`
- ☐ All Phase 2 code committed to `feature/data-collection` branch

- [ ] Branch merged to `dev` after review

> [!CAUTION] **Do NOT proceed to Phase 3 until entity resolution (Task 2C) and validation (Task 2D) are complete.** Building a graph on unresolved entities will produce a broken topology — duplicate nodes, disconnected edges, and wrong adjacency matrix. This will cascade into garbage GNN predictions in Phase 4 and meaningless contagion simulations in Phase 5. Entity resolution is the **foundation** — get it right before moving forward.

Estimated Time Allocation

Task	Days	Team Member	Dependencies
2A: Manual SEC extraction	Days 1–3	ML Person	Phase 1 anchor list
2A: NER pipeline	Days 3–7	ML Person	Step 2A-1 (need examples to design NLP)
2A: Supplementary sources	Days 3–7	Both	None (parallel)
2B: yfinance pull	Days 1–3	ML Person	Phase 1 ticker list
2B: Z-Score computation	Days 3–4	ML Person	Step 2B-1 complete
2B: Private company data	Days 4–5	Both	Identified private companies
2C: Entity resolution	Days 5–8	ML Person	Tasks 2A + 2B initial output

2D: Validation	Days 8–10	Both	Task 2C complete
2E: SQLite setup	Days 1–4	Developer	Phase 1 schema definition
2F: FastAPI skeleton	Days 1–2	Developer	None

> [!TIP] **The Developer can start Tasks 2E and 2F on Day 1** while the ML Person works on data collection (2A, 2B). These tracks are independent and converge at Task 2D. This parallelism is how you finish Phase 2 in two weeks despite the amount of work.